

DEEP COMPUTER VISION (2): CNN DAN VARIASINYA

AlexNet, VGG, GoogLeNet, ResNet, MobileNet, EfficientNet, ConvNeXt, dan
Deteksi/Segmentasi

Setelah mengikuti pertemuan ini, mahasiswa diharapkan mampu:

- 1 Menjelaskan evolusi CNN dari AlexNet menuju VGG**
- 2 Menjelaskan desain blok konvolusi berlapis (3×3 bertumpuk) pada VGG**
- 3 Menjelaskan residual connection dan alasan ResNet bisa jauh lebih dalam**
- 4 Membandingkan efisiensi MobileNet (depthwise separable) dengan CNN standar**
- 5 Mengidentifikasi elemen modern ConvNeXt yang diadopsi dari Transformer**
- 6 Menjelaskan prinsip object detection dan segmentasi sebagai perluasan CNN**

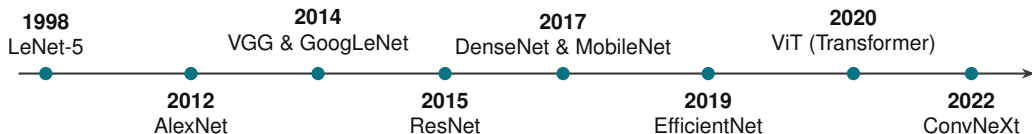
OUTLINE

- 1 Era ImageNet: AlexNet & VGG
- 2 Lebih Dalam dengan Residual: ResNet
- 3 CNN Efisien: MobileNet & EfficientNet
- 4 ConvNeXt: CNN untuk Era 2020-an
- 5 Perbandingan Arsitektur
- 6 Dari Klasifikasi ke Deteksi & Segmentasi

Temuan Notebook 03 (Pertemuan 4)

LeNet-5 mencapai $> 98\%$ pada MNIST, tetapi **mentok di kisaran $\sim 60\%$ pada CIFAR-10**: kapasitas kecil (6/16 filter), hanya 2 layer konvolusi, tanpa BatchNorm/Dropout/augmentasi.

Pertanyaan hari ini: bagaimana CNN menjadi **lebih dalam, lebih efisien, dan lebih serbaguna**?



Tiga tema besar: **kedalaman** (AlexNet $\rightarrow$ VGG $\rightarrow$ ResNet) · **efisiensi** (GoogLeNet, MobileNet, EfficientNet) · **desain modern** (ConvNeXt).

BAGIAN 01 · ERA IMAGENET

AlexNet & VGG

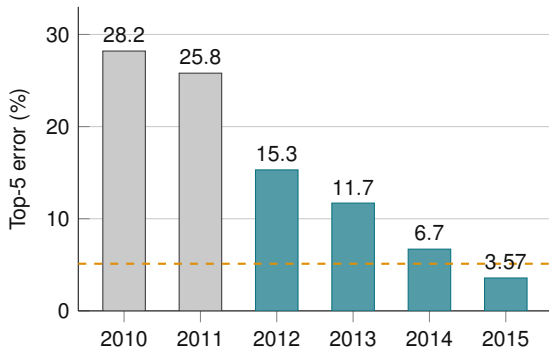
Lebih Dalam, Lebih Besar, dan Dilatih dengan GPU

ImageNet & Kompetisi ILSVRC

Benchmark yang Memicu Revolusi

ILSVRC (2010–2017)

- $\pm 1,2$ juta citra latih, **1.000 kelas**
- Metrik: **top-5 error**
- **2012:** AlexNet memangkas error ~ 10 poin sekaligus.
- Sejak itu seluruh pemenang adalah **CNN**.
- 2015: ResNet melampaui estimasi manusia ($\sim 5,1\%$).



2010 NEC · 2011 XRCE · 2012 AlexNet · 2013 ZFNet · 2014
GoogLeNet · 2015 ResNet · garis putus-putus: estimasi
manusia $\approx 5,1\%$

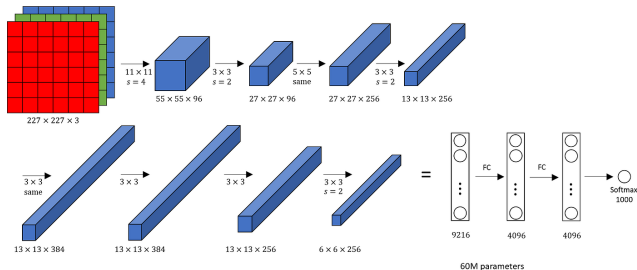
AlexNet (2012): Tim SuperVision

Krizhevsky, Sutskever & Hinton



Credit: University of Toronto / Wired

- Top-5 error **15,3%** vs 26,2%
- 5 conv + 3 FC = **8 layer**
- ~6 hari di **2 GPU GTX 580**



Credit: Krizhevsky et al. (2012), diolah ulang

Arsitektur terbelah dua jalur karena model tidak muat di memori satu GPU — ide yang kelak kembali sebagai **grouped convolution**.

Menelusuri Dimensi & Parameter AlexNet

Menerapkan Rumus Pertemuan 4

Layer	Konfigurasi	Output	Param
Input	RGB	$227^2 \times 3$	–
Conv1	$96, 11 \times 11, S=4$	$55^2 \times 96$	34.944
MaxPool	$3 \times 3, S=2$	$27^2 \times 96$	0
Conv2	$256, 5 \times 5, P=2$	$27^2 \times 256$	614.656
MaxPool	$3 \times 3, S=2$	$13^2 \times 256$	0

Contoh: Output Conv1

$$W_{out} = \left\lfloor \frac{227 - 11 + 2 \cdot 0}{4} \right\rfloor + 1 = 55$$

Layer	Konfigurasi	Output	Param
Conv3–5	$384/384/256, 3 \times 3$	$13^2 \times 256$	3,7 jt
MaxPool	$3 \times 3, S=2$	$6^2 \times 256$	0
FC6	$9216 \rightarrow 4096$	4096	37,8 jt
FC7	$4096 \rightarrow 4096$	4096	16,8 jt
FC8	$4096 \rightarrow 1000$	1000	4,1 jt

- Total $\approx$ **61 juta** parameter — $\sim 1000 \times$ LeNet-5 (61.706).
- $\sim$ **96%** parameter ada di FC6–FC8 (58,6 juta), pola yang sama dengan LeNet-5.
- FC6 saja: $(6 \cdot 6 \cdot 256 + 1) \times 4096 = 37,8$ juta.

1. Aktivasi ReLU $f(x) = \max(0, x)$

- Sigmoid: $\sigma'(x) \leq 0,25 \Rightarrow$ gradien mengecil tiap layer (*vanishing gradient*).
- ReLU: turunan = 1 untuk $x > 0 \Rightarrow$ konvergensi $\sim 6\times$ lebih cepat daripada tanh.

2. Dropout ($p = 0,5$ di FC6 & FC7)

- Mematikan neuron acak saat training $\Rightarrow$ mencegah ko-adaptasi, menekan overfitting 58 juta bobot FC.

3. Augmentasi Data

- Random crop 224×224 dari 256×256 + horizontal flip ($\sim 2048\times$ variasi).
- Perturbasi warna berbasis PCA pada kanal RGB.

4. Training di GPU

- Konvolusi paralel di CUDA; model dibagi ke 2 GPU.
- Bukti bahwa **data besar + GPU** membuka jalan deep learning.

(Juga: Local Response Normalization & overlapping pooling — kini sudah ditinggalkan.)

LeNet-5 vs AlexNet: Lompatan Generasi

14 Tahun, Skala Berubah Total

Aspek	LeNet-5 (1998)	AlexNet (2012)
Layer terlatih	5 (2 conv + 3 FC)	8 (5 conv + 3 FC)
Parameter	61.706	~61 juta
Input	$32 \times 32 \times 1$	$227 \times 227 \times 3$
Aktivasi	Tanh	ReLU
Pooling	Average 2×2	Max 3×3 , stride 2 (overlap)
Regularisasi	—	Dropout + augmentasi
Dataset	MNIST (60 ribu, 10 kelas)	ImageNet (1,2 juta, 1000 kelas)
Perangkat	CPU	2× GPU GTX 580

Takeaway: ide dasarnya sama ($conv \rightarrow pool \rightarrow FC$); yang berubah adalah **skala data, komputasi, dan trik pelatihan**.

```
import torch.nn as nn
from torchvision import models

features = nn.Sequential(
    nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
    nn.ReLU(inplace=True),          # bukan Tanh
    nn.MaxPool2d(kernel_size=3, stride=2), # overlapping
    nn.Conv2d(64, 128, kernel_size=5, padding=2),
    nn.ReLU(inplace=True),
    # ... conv3-conv5, lalu AdaptiveAvgPool2d((6, 6))
)
alexnet = models.alexnet(weights="IMAGENET1K_V1")
```

- Versi torchvision memakai 64 filter di conv1 (varian “one weird trick”, 61,1 juta parameter).
- `AdaptiveAvgPool2d((6,6))` menjamin input FC selalu $6 \times 6 \times 256$ walau resolusi input berbeda.
- Dropout diletakkan sebelum FC6 & FC7 di classifier.

Hipotesis VGG

Pakai **hanya filter** 3×3 (stride 1, padding 1) dan tumpuk sebanyak mungkin. Kedalaman 16–19 layer, desain seragam.

Generalisasi insight RF Pertemuan 4 (C kanal masuk & keluar):

Tumpukan	RF	Bobot
$1 \times (5 \times 5)$	5×5	$25C^2$
$2 \times (3 \times 3)$	5×5	$18C^2$
$1 \times (7 \times 7)$	7×7	$49C^2$
$3 \times (3 \times 3)$	7×7	$27C^2$

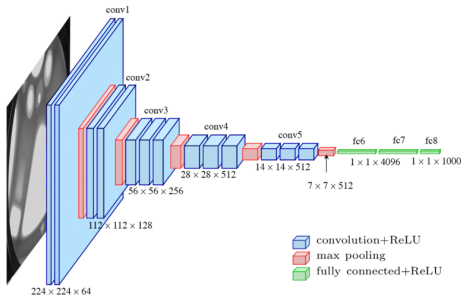
Keuntungan filter kecil bertumpuk:

- Receptive field sama, **bobot 28–45% lebih hemat**.
- Lebih banyak **non-linearitas** (ReLU di setiap layer) $\Rightarrow$ fungsi keputusan lebih diskriminatif.
- Desain modular: cukup atur *jumlah conv per blok* dan *jumlah kanal*.

Contoh $C = 256$: $49C^2 = 3,2$ juta vs $27C^2 = 1,8$ juta bobot.

Arsitektur VGG-16

5 Blok Konvolusi + 3 FC



Credit: Simonyan & Zisserman (2015), ilustrasi via Medium

Tiap blok: max-pool 2×2 (resolusi $\div 2$), kanal $\times 2$ hingga 512 — **spasial mengecil, kanal membesar.**

Input $224 \times 224 \times 3$

Blok 1: $2 \times \text{conv } 64 \rightarrow 112^2$

Blok 2: $2 \times \text{conv } 128 \rightarrow 56^2$

Blok 3: $3 \times \text{conv } 256 \rightarrow 28^2$

Blok 4: $3 \times \text{conv } 512 \rightarrow 14^2$

Blok 5: $3 \times \text{conv } 512 \rightarrow 7^2$

FC 4096 $\rightarrow$ FC 4096 $\rightarrow$ FC 1000

Softmax (1000 kelas)

Resolusi ditulis setelah max-pool tiap blok.

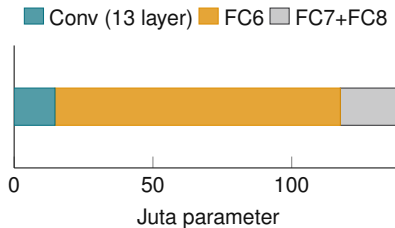
Di Mana 138 Juta Parameter VGG-16?

Analisis Distribusi Bobot

Contoh: FC6 (flatten $7 \times 7 \times 512$ ke 4096)

$$7 \cdot 7 \cdot 512 \cdot 4096 = \mathbf{102,8 \text{ juta bobot}}$$

- Seluruh 13 layer konvolusi: **14,7 juta** (10,6%)
- 3 layer FC: **123,6 juta** (89,4%)
- Namun komputasi terbesar ada di conv: **15,5 GFLOPs** per citra.



Keterbatasan VGG

Bobot >500 MB, lambat di inferensi. Dorongan berikutnya: **buang FC raksasa** (GoogLeNet: Global Average Pooling).

```
def vgg_block(c_in, c_out, n_conv):
    layers = []
    for i in range(n_conv):
        c = c_in if i == 0 else c_out
        layers += [nn.Conv2d(c, c_out, 3, padding=1),
                   nn.ReLU(inplace=True)]
    layers.append(nn.MaxPool2d(2, 2))      # resolusi / 2
    return nn.Sequential(*layers)

# VGG-16: (jumlah conv, kanal) per blok
cfg = [(2, 64), (2, 128), (3, 256), (3, 512), (3, 512)]
```

- $\text{padding}=1 + 3 \times 3 = \text{same conv}$: resolusi hanya berubah di pooling.
- VGG-19 cukup mengubah jumlah conv per blok menjadi (2, 2, 4, 4, 4).
- Varian vgg16_bn menyisipkan BatchNorm $\Rightarrow$ top-1 naik 71,6% $\rightarrow$ 73,4%.

Ide: 1×1 conv sebagai reduksi kanal (*bottleneck*)

Konvolusi 1×1 adalah FC layer per piksel: mencampur kanal tanpa mengubah resolusi.

Contoh: input $28 \times 28 \times 192 \rightarrow 32$ filter 5×5 (jumlah perkalian):

Langsung:

$$28 \cdot 28 \cdot 32 \cdot (5 \cdot 5 \cdot 192) \approx \mathbf{120,4 \text{ juta}}$$

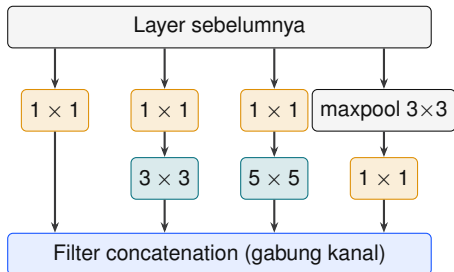
Via 1×1 ke 16 kanal dulu:

$$\underbrace{28^2 \cdot 16 \cdot 192}_{2,4 \text{ jt}} + \underbrace{28^2 \cdot 32 \cdot (25 \cdot 16)}_{10,0 \text{ jt}} \approx \mathbf{12,4 \text{ juta}}$$

- Hemat $\sim 10\times$ komputasi — teknik ini dipakai ulang di ResNet bottleneck, MobileNet, dan ConvNeXt.

Modul Inception

Multi-skala dalam Satu Blok



Oranye: 1×1 reduksi kanal · Teal: filter spasial 3×3 / 5×5

- 4 cabang paralel: objek bisa kecil (1×1 , 3×3) atau besar (5×5) — **biarkan jaringan memilih.**
- GoogLeNet: 22 layer, 9 modul Inception, juara ILSVRC 2014 (top-5 **6,7%**).
- Mengganti FC raksasa dengan **Global Average Pooling**: hanya **~6,6 juta** parameter (vs 138 juta VGG-16).

BAGIAN 02 · RESIDUAL

ResNet & Kawan-kawan

BatchNorm, Degradation Problem, Skip Connection, DenseNet

Recap: Batch Normalization

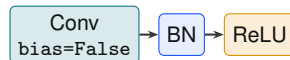
Prasyarat Melatih Jaringan Dalam (P2)

Untuk setiap kanal pada mini-batch B

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

- γ, β dipelajari; saat `eval()` memakai *running mean/var*.
- Menstabilkan distribusi aktivasi $\Rightarrow$ learning rate lebih besar, tidak sensitif inisialisasi.

Pola baku CNN modern:



- Bias conv redundan karena β BN sudah menggeser.
- Tanpa BN, VGG harus dilatih bertahap (11 $\rightarrow$ 16 layer); dengan BN, ResNet-152 bisa dilatih langsung.

Degradation Problem

Mengapa Tidak Sekadar Menumpuk Layer?

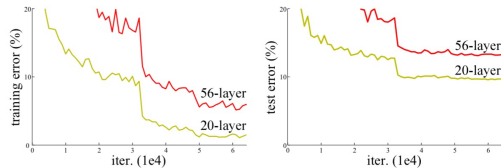


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

Credit: He et al. (2016), CIFAR-10 plain network 20 vs 56 layer

Pengamatan He et al. (2016)

Jaringan *plain* 56 layer memiliki **training error** lebih tinggi daripada 20 layer.

- **Bukan overfitting** — error latih juga naik.
- Secara teori, model dalam minimal bisa menyamai model dangkal (layer tambahan = identitas). Namun optimizer **sulit mempelajari identitas**.

ImageNet top-1 err.	18 layer	34 layer
Plain	27,94%	28,54%
ResNet	27,88%	25,03%

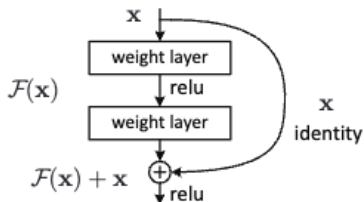


Figure 2. Residual learning: a building block.

Credit: He et al. (2016)

Bila pemetaan optimal mendekati identitas, cukup dorong $F(x) \rightarrow 0$ — jauh lebih mudah daripada mempelajari $H(x) = x$ dari nol.

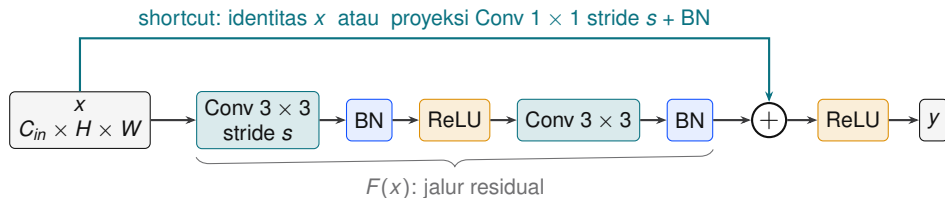
Formulasi Residual

$$H(x) = F(x) + x \implies F(x) = H(x) - x$$

Aliran gradien (“jalan tol”):

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \left(1 + \frac{\partial F}{\partial x} \right)$$

- Suku $+1$: gradien selalu punya jalur langsung ke layer awal, tidak hilang walau $\partial F / \partial x$ kecil.
- Tanpa parameter tambahan dan hampir tanpa komputasi tambahan (hanya penjumlahan).



Identity shortcut ($s = 1$, $C_{in} = C_{out}$)

- $y = \text{ReLU}(F(x) + x)$, nol parameter tambahan.

Projection shortcut (awal tiap stage)

- Resolusi $\div 2$ & kanal $\times 2$: x disesuaikan dengan Conv 1×1 , stride 2.

Basic Block di PyTorch

Satu Baris yang Mengubah Segalanya

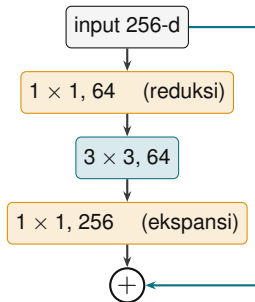
```
class BasicBlock(nn.Module):
    def __init__(self, c_in, c_out, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(c_in, c_out, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(c_out)
        self.conv2 = nn.Conv2d(c_out, c_out, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(c_out)
        self.shortcut = nn.Identity()
        if stride != 1 or c_in != c_out:    # proyeksi
            self.shortcut = nn.Sequential(
                nn.Conv2d(c_in, c_out, 1, stride, bias=False),
                nn.BatchNorm2d(c_out))

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return F.relu(out + self.shortcut(x))    # <-- skip
```

- Hapus +
 $\text{self.shortcut}(x) \Rightarrow$
kembali menjadi **plain network**.
- Eksperimen semester lalu: *Plain-34* $\rightarrow$ *ResNet-34* hanya dengan menambahkan penjumlahan ini.
- Uji:
 $\text{BasicBlock}(64, 128, 2)$
mengubah
 $56^2 \times 64 \rightarrow 28^2 \times 128$.

Bottleneck Block (ResNet-50/101/152)

Reduksi 1×1 ala GoogLeNet



Pola **lebar** $\rightarrow$ **sempit** $\rightarrow$ **lebar**; BN + ReLU setelah tiap conv (tidak digambar).

Contoh: bobot satu blok pada 256 kanal

Basic: $2 \cdot (3 \cdot 3 \cdot 256 \cdot 256) = 1.179.648$

Bottleneck: $16.384 + 36.864 + 16.384 = \mathbf{69.632}$

- $\sim 17\times$ lebih hemat $\Rightarrow$ blok bisa ditumpuk jauh lebih banyak.
- ResNet-50 (bottleneck) vs ResNet-34 (basic): 25,6 vs 21,8 juta parameter, tetapi top-1 **76,1%** vs 73,3%.

Konfigurasi ResNet-18/34/50

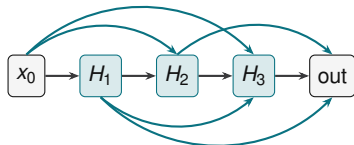
Empat Stage, Resolusi $\div 2$ per Stage

Stage	Output	ResNet-18	ResNet-34	ResNet-50
conv1	112×112	$7 \times 7, 64, \text{stride } 2$ + maxpool $3 \times 3, \text{stride } 2$		
conv2_x	56×56	$[3 \times 3, 64] \times 2$	$[3 \times 3, 64] \times 3$	$[1 \times 1, 64 \mid 3 \times 3, 64 \mid 1 \times 1, 256] \times 3$
conv3_x	28×28	$[3 \times 3, 128] \times 2$	$[3 \times 3, 128] \times 4$	$[1 \times 1, 128 \mid 3 \times 3, 128 \mid 1 \times 1, 512] \times 4$
conv4_x	14×14	$[3 \times 3, 256] \times 2$	$[3 \times 3, 256] \times 6$	$[1 \times 1, 256 \mid 3 \times 3, 256 \mid 1 \times 1, 1024] \times 6$
conv5_x	7×7	$[3 \times 3, 512] \times 2$	$[3 \times 3, 512] \times 3$	$[1 \times 1, 512 \mid 3 \times 3, 512 \mid 1 \times 1, 2048] \times 3$
head	1×1	Global Average Pooling + FC 1000		
Parameter / GFLOPs		11,7 jt / 1,8	21,8 jt / 3,7	25,6 jt / 4,1
ImageNet top-1		69,8%	73,3%	76,1%

- Downsampling memakai **conv stride 2** di awal stage (bukan pooling), lalu **GAP** menggantikan FC raksasa VGG.
- ResNet-101/152: pola bottleneck yang sama, conv4_x diulang 23/36 kali.

DenseNet (2017): Gabungkan, Jangan Jumlahkan

Dense Connectivity



Setiap layer menerima semua fitur sebelumnya

$$x_\ell = H_\ell([x_0, x_1, \dots, x_{\ell-1}])$$

- **Growth rate** k (mis. 32): tiap layer hanya menambah k kanal baru $\Rightarrow$ layer sempit, fitur dipakai ulang.
- **Transition layer** (1×1 conv + avgpool 2×2) mengontrol ledakan kanal antar blok.
- Hemat parameter, namun boros memori aktivasi saat training.

	ResNet	DenseNet
Operasi	$x + F(x)$	$[x_0, \dots, x_{\ell-1}]$ (concat)
Params	25,6 jt (R-50)	8,0 jt (D-121)
Top-1	76,1%	74,4%

BAGIAN 03 · EFISIENSI

MobileNet & EfficientNet

Depthwise Separable Convolution, Inverted Residual, Compound Scaling

Standar vs Depthwise Separable

Memecah Satu Konvolusi Menjadi Dua

Motivasi: ponsel, drone, kamera IoT — latensi & energi terbatas. ResNet-50 butuh 4,1 GFLOPs per citra.

Notasi: kernel $D_K \times D_K$, kanal masuk M , kanal keluar N , feature map $D_F \times D_F$.

Konvolusi standar

$$D_K^2 \cdot M \cdot N \cdot D_F^2$$

Filter spasial **dan** pencampuran kanal sekaligus.

Depthwise + Pointwise

$$\underbrace{D_K^2 \cdot M \cdot D_F^2}_{\text{depthwise}} + \underbrace{M \cdot N \cdot D_F^2}_{\text{pointwise } 1 \times 1}$$

Filter spasial per kanal, lalu campur kanal.

Rasio biaya:

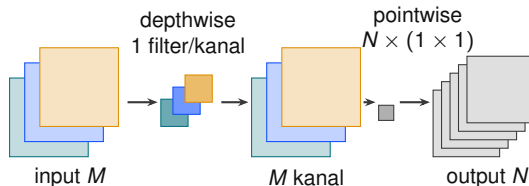
$$\frac{\text{separable}}{\text{standar}} = \frac{1}{N} + \frac{1}{D_K^2} \Rightarrow \text{untuk } 3 \times 3 \text{ dan } N \text{ besar: } \approx \frac{1}{9} \text{ (hemat 8-9}\times\text{)}$$

Contoh Numerik & Visualisasi

$$3 \times 3, M = N = 256, D_F = 56$$

	Mult-adds	Bobot
Standar	1,85 G	589.824
Depthwise	0,007 G	2.304
Pointwise	0,206 G	65.536
Separable	0,21 G	67.840

- Hemat $8,7\times$, sesuai rumus $\frac{1}{256} + \frac{1}{9} \approx 0,115$.
- $\sim 97\%$ biaya kini ada di pointwise 1×1 (operasi matriks yang sangat efisien di hardware).



Depthwise: tiap kanal difilter sendiri (tanpa mencampur). Pointwise: N filter 1×1 mencampur informasi antar kanal.

MobileNetV1 (2017) di PyTorch

Kuncinya: Argumen `groups`

```
def dw_separable(c_in, c_out, stride=1):  
    return nn.Sequential(  
        # depthwise: groups=c_in -> 1 filter per kanal  
        nn.Conv2d(c_in, c_in, 3, stride, 1,  
                  groups=c_in, bias=False),  
        nn.BatchNorm2d(c_in), nn.ReLU(inplace=True),  
        # pointwise: konvolusi 1x1 mencampur kanal  
        nn.Conv2d(c_in, c_out, 1, bias=False),  
        nn.BatchNorm2d(c_out), nn.ReLU(inplace=True))
```

Cek: bobot conv `dw_separable(256,256)` = 67.840 vs
`nn.Conv2d(256,256,3)` = 589.824.

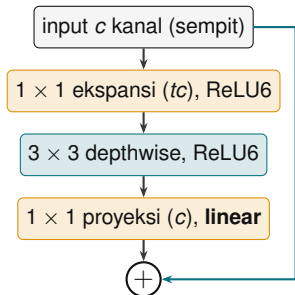
Dua “kenop” efisiensi MobileNet:

- **Width multiplier**
 $\alpha \in \{1; 0,75; 0,5; 0,25\}$:
kanal $M, N \rightarrow \alpha M, \alpha N$.
- **Resolution multiplier ρ** :
input
 $224 \rightarrow 192 \rightarrow 160 \rightarrow 128$.
- Biaya turun $\approx \alpha^2 \rho^2$.

MobileNetV1: 4,2 juta
parameter, 0,57 G mult-adds,
top-1 70,6% — mendekati
VGG-16 dengan komputasi
 $\sim 27\times$ lebih kecil.

MobileNetV2 (2018): Inverted Residual

Sempit $\rightarrow$ Lebar $\rightarrow$ Sempit



Faktor ekspansi $t = 6$; skip hanya bila stride 1 & $C_{in} = C_{out}$.

	ResNet bottleneck	MBv2 inverted
Bentuk	lebar–sempit–lebar	sempit–lebar–sempit
Conv tengah	3×3 standar	3×3 depthwise
Skip antar	tensor lebar	tensor sempit
Aktivasi akhir	ReLU	linear

- **Linear bottleneck:** ReLU pada tensor sempit membuang informasi, maka proyeksi dibiarkan linear.
- MobileNetV2: 3,5 juta parameter, 0,30 GFLOPs, top-1 **71,9%**. Blok ini menjadi fondasi EfficientNet & ConvNeXt.

Masalah: memperbesar model biasanya hanya satu dimensi — lebih dalam (ResNet-152), lebih lebar (WideResNet), atau resolusi lebih tinggi. Hasilnya cepat jenuh.

Compound scaling dengan koefisien ϕ

$$d = \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi \\ \text{s.t. } \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

$\text{FLOPs} \propto d \cdot w^2 \cdot r^2$, sehingga tiap $\phi + 1$ menggandakan FLOPs. Hasil grid search: $\alpha = 1,2$, $\beta = 1,1$, $\gamma = 1,15$.

- **Baseline B0:** dicari dengan Neural Architecture Search; blok MBConv (inverted residual MobileNetV2 + Squeeze-and-Excitation).
- B0 $\rightarrow$ B7: naikkan ϕ saja.

Model	Params	Input	Top-1
EfficientNet-B0	5,3 jt	224	77,7%
EfficientNet-B7	66 jt	600	84,3%
ResNet-50	25,6 jt	224	76,1%

B0 lebih akurat dari ResNet-50 dengan $\sim 5\times$ lebih sedikit parameter.

BAGIAN 04 · CNN MODERN

ConvNeXt

“A ConvNet for the 2020s”: Memodernisasi ResNet dengan Resep Transformer

2020–2021: Transformer masuk ke Vision

ViT (2020) dan Swin Transformer (2021) melampaui ResNet di ImageNet; muncul narasi “CNN akan tergantikan”.

- Namun Swin juga memakai **jendela lokal** dan **hierarki stage** — mirip CNN.
- Selain arsitektur, Transformer datang dengan **resep training baru**: AdamW, 300 epoch, Mixup/CutMix, RandAugment, label smoothing, stochastic depth.

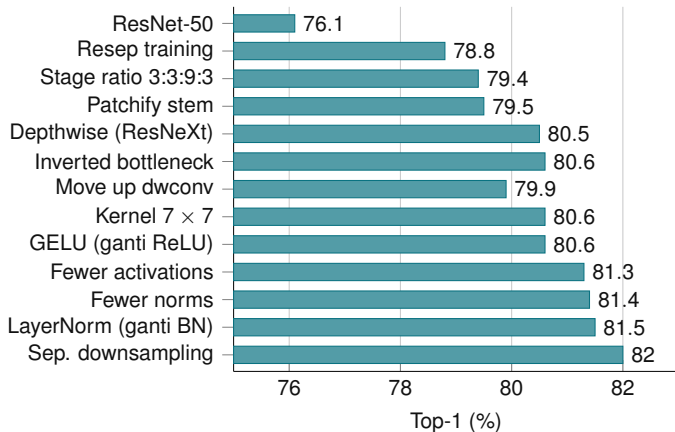
Pertanyaan Liu et al. (Meta AI, 2022)

Bila ResNet-50 dimodernisasi **langkah demi langkah** memakai pilihan desain Transformer, seberapa jauh CNN murni bisa bersaing?

Model	GFLOPs	Top-1
ResNet-50 (resep lama)	4,1	76,1%
Swin-T	4,5	81,3%
ConvNeXt-T	4,5	82,1%

Roadmap: ResNet-50 → ConvNeXt-T

Kontribusi Tiap Modifikasi (ImageNet Top-1)



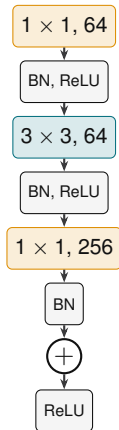
- **Resep training** saja: +2,7 poin — sebagian “keunggulan Transformer” ternyata dari cara melatihnya.
- **Makro**: stage ratio ala Swin, stem 4×4 stride 4 (*patchify*).
- **Blok**: depthwise, inverted bottleneck, kernel 7×7 .
- **Mikro**: GELU, lebih sedikit aktivasi & normalisasi, LayerNorm.

Sumber: Liu et al. (2022), Fig. 2.

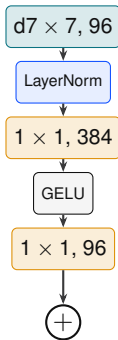
Blok ResNet vs Blok ConvNeXt

Elemen yang Diadopsi dari Transformer

ResNet bottleneck



ConvNeXt block



Padanan dengan Transformer:

- **Patchify stem** $4 \times 4/4 \leftrightarrow$ patch embedding ViT.
- **Depthwise** $7 \times 7 \leftrightarrow$ pencampuran spasial per kanal (seperti self-attention per jendela Swin).
- 1×1 **ekspansi** $4 \times \leftrightarrow$ MLP/FFN $4 \times$.
- **1 GELU + 1 LN** per blok, seperti blok Transformer.
- Downsampling terpisah (LN + conv $2 \times 2/2$) di antara stage.

ConvNeXt Block di PyTorch

Versi Ringkas (tanpa Layer Scale)

```
class ConvNeXtBlock(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.dwconv = nn.Conv2d(dim, dim, 7, padding=3, groups=dim)
        self.norm = nn.LayerNorm(dim) # channels-last
        self.pw1 = nn.Linear(dim, 4 * dim) # 1x1, ekspansi 4x
        self.act = nn.GELU()
        self.pw2 = nn.Linear(4 * dim, dim) # 1x1, proyeksi

    def forward(self, x): # (B, C, H, W)
        y = self.dwconv(x).permute(0, 2, 3, 1) # (B, H, W, C)
        y = self.pw2(self.act(self.pw1(self.norm(y))))
        return x + y.permute(0, 3, 1, 2) # residual
```

- `nn.Linear` pada tensor *channels-last* = konvolusi 1×1 .
- Mirip inverted bottleneck MobileNetV2, tetapi depthwise dipindah ke **awal** blok.
- ConvNeXt-T: stage (3, 3, 9, 3), kanal (96, 192, 384, 768), 28,6 juta parameter.
- Versi resmi menambah *layer scale & stochastic depth*.

BAGIAN 05 · PERBANDINGAN

Perbandingan Arsitektur

Akurasi, Parameter, dan Komputasi pada ImageNet

Tabel Perbandingan pada ImageNet-1K

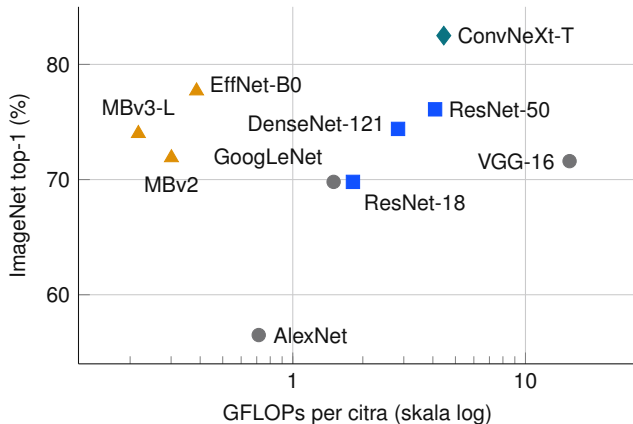
Bobot Pretrained torchvision

Model	Tahun	Params (jt)	GFLOPs	Top-1 (%)	Ide kunci
AlexNet	2012	61,1	0,71	56,5	ReLU, dropout, GPU
VGG-16	2014	138,4	15,47	71,6	tumpukan 3×3
GoogLeNet	2014	6,6	1,50	69,8	Inception, 1×1 , GAP
ResNet-18	2015	11,7	1,81	69,8	residual (basic block)
ResNet-50	2015	25,6	4,09	76,1	residual (bottleneck)
DenseNet-121	2017	8,0	2,83	74,4	dense concatenation
MobileNetV2	2018	3,5	0,30	71,9	inverted residual
MobileNetV3-L	2019	5,5	0,22	74,0	NAS + SE + h-swish
EfficientNet-B0	2019	5,3	0,39	77,7	compound scaling
ConvNeXt-T	2022	28,6	4,46	82,5	modernisasi ResNet

Angka dari dokumentasi `torchvision.models` (evaluasi single-crop 224×224) — sama dengan bobot yang akan dipakai pada Studio Proyek 2 (Pertemuan 6).

Observasi: VGG-16 memakai $40\times$ parameter dan $70\times$ komputasi MobileNetV2 untuk akurasi yang hampir sama.

Akurasi vs Komputasi



Legenda:

- Klasik (2012–2014)
- Residual / dense
- ▲ Efisien (mobile)
- ◆ Modern (2022)

Takeaway: kemajuan arsitektur bukan sekadar “lebih besar”. Model di kiri atas (EfficientNet, MobileNetV3) mencapai akurasi lebih tinggi dengan komputasi $\sim 40\times$ lebih kecil daripada VGG.

Kebutuhan	Kandidat
Baseline umum	ResNet-18 / 50
Edge / mobile	MobileNetV3, EffNet-B0
Akurasi maksimum	ConvNeXt-T / S
Parameter minim	DenseNet-121
Pembandingan historis	AlexNet, VGG-16

```
from torchvision import models
m = models.resnet50(weights="IMAGENET1K_V1")
m.fc = nn.Linear(m.fc.in_features, 5)
# head lain -> MobileNetV3: m.classifier[3]
#                  ConvNeXt : m.classifier[2]
```

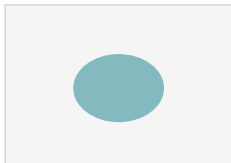
- Semua backbone di atas tersedia **pretrained** di `torchvision.models`.
- Pertemuan 6: bandingkan backbone & strategi fine-tuning (freezing, linear probing vs full fine-tuning).

BAGIAN 06 · DETEKSI & SEGMENTASI

Beyond Classification

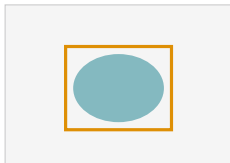
Format Bounding Box, IoU, mAP, R-CNN, YOLO, FCN, U-Net, Mask R-CNN

Klasifikasi



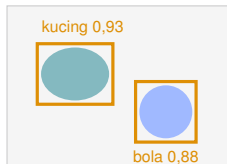
1 label kelas

Lokalisasi



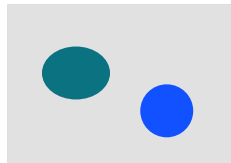
label + 1 bbox

Deteksi Objek



$N \times (\text{bbox}, \text{label}, \text{skor})$

Segmentasi



label per piksel

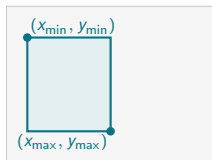
- **Deteksi:** jumlah objek tidak diketahui $\Rightarrow$ output berukuran variabel.
- **Semantic segmentation:** kelas per piksel, tidak membedakan instans.

- **Instance segmentation:** mask terpisah untuk setiap objek (kucing #1 vs #2).
- Semua tugas memakai **backbone CNN** yang sama dengan klasifikasi.

Tiga Konvensi Penulisan Bounding Box

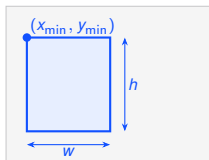
Pascal VOC · COCO · YOLO

Pascal VOC



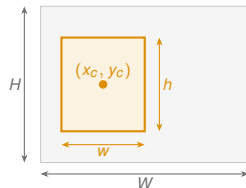
$[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$

COCO



$[x_{\min}, y_{\min}, w, h]$

YOLO



$[c, x_c/W, y_c/H, w/W, h/H]$

	Pascal VOC	COCO	YOLO
Parameter	2 sudut (xyxy)	sudut kiri-atas + ukuran (xywh)	pusat + ukuran (cxcywh)
Satuan	piksel	piksel	relatif $[0, 1]$ (dibagi W, H)
Berkas	1 XML per citra	1 JSON per split	1 .txt per citra
Pemakai	target torchvision	pycocotools (mAP)	Ultralytics

Titik asal $(0, 0)$ di **kiri-atas** citra, sumbu y ke bawah (sama untuk ketiganya).

Contoh Konversi Antar Format

Box yang Sama, Tiga Cara Menulis

Contoh: citra 640×480 , kelas 0

Pascal VOC	64, 96, 320, 384
COCO	64, 96, 256, 288
YOLO	0 0.300 0.500 0.400 0.600

$$w = 320 - 64 = 256, \quad h = 384 - 96 = 288$$

$$x_c = \frac{64+320}{2} = 192 \Rightarrow 192/640 = 0,3$$

$$y_c = \frac{96+384}{2} = 240 \Rightarrow 240/480 = 0,5$$

$$w/W = 256/640 = 0,4,$$

$$h/H = 288/480 = 0,6$$

Rumus konversi dari xyxy

$$\text{COCO: } w = x_{\max} - x_{\min}, \quad h = y_{\max} - y_{\min}$$

$$\text{YOLO: } \hat{x}_c = \frac{x_{\min} + x_{\max}}{2W}, \quad \hat{y}_c = \frac{y_{\min} + y_{\max}}{2H}$$

$$\hat{w} = \frac{w}{W}, \quad \hat{h} = \frac{h}{H}$$

$$\text{Balik: } x_{\min} = (\hat{x}_c - \frac{\hat{w}}{2}) W, \quad y_{\min} = (\hat{y}_c - \frac{\hat{h}}{2}) H$$

Jebakan umum:

- **Salah baca format:** box COCO di atas dibaca sebagai xyxy $\Rightarrow$ IoU dengan box asli hanya **0,5**, padahal model benar.
- **Resize:** label VOC/COCO ikut diskalakan; label YOLO tetap (kecuali crop/padding).
- Anotasi VOC asli berindeks piksel mulai 1.

Pascal VOC — Annotations/img001.xml

```
<object>
  <name>cat</name>
  <bndbox>
    <xmin>64</xmin>  <ymin>96</ymin>
    <xmax>320</xmax> <ymax>384</ymax>
  </bndbox>
</object>
```

COCO — annotations/instances_train.json

```
"annotations": [{"id": 1, "image_id": 1,
  "category_id": 17, "iscrowd": 0,
  "bbox": [64, 96, 256, 288]}]
```

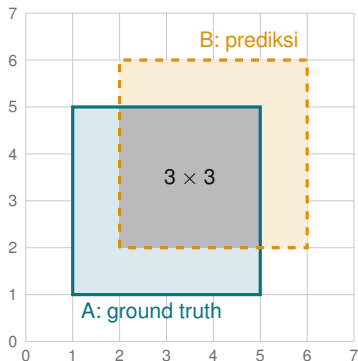
YOLO — labels/img001.txt

```
0 0.300 0.500 0.400 0.600
```

```
import torch
from torchvision.ops import box_convert

W, H = 640, 480
voc = torch.tensor([[64., 96., 320., 384.]]) # xyxy
coco = box_convert(voc, "xyxy", "xywh")
# tensor([[ 64.,  96., 256., 288.]])
cxcy = box_convert(voc, "xyxy", "cxcywh")
yolo = cxcy / torch.tensor([W, H, W, H])
# tensor([[0.3000, 0.5000, 0.4000, 0.6000]])
```

- `box_convert` bekerja dalam piksel; normalisasi YOLO dilakukan manual.
- `Albumentations BboxParams(format=...)`: box ikut berubah saat augmentasi.
- Kelas: COCO `category_id` (kucing = 17); YOLO indeks mulai 0.



Box dalam format `xyxy` (Pascal VOC) + label kelas + skor keyakinan.

Intersection over Union

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \in [0, 1]$$

Contoh: $A = (1, 1, 5, 5)$, $B = (2, 2, 6, 6)$

Irisan = $3 \times 3 = 9$, gabungan = $16 + 16 - 9 = 23$
 $\text{IoU} = 9/23 \approx \mathbf{0,39} < 0,5 \Rightarrow$ dihitung **salah (FP)**
pada ambang 0,5.

Non-Maximum Suppression (per kelas)

- 1 Urutkan box berdasarkan skor.
- 2 Ambil box skor tertinggi → simpan.
- 3 Buang box lain dengan $\text{IoU} > \text{ambang}$ (mis. 0,5) terhadap box tersebut.
- 4 Ulangi sampai tidak ada box tersisa.

Detektor menghasilkan ratusan box yang tumpang tindih untuk satu objek; NMS menyisakan satu.

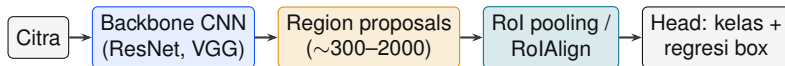
Evaluasi deteksi:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

- Prediksi = TP bila $\text{IoU} \geq \text{ambang}$ & kelas benar.
- **AP**: luas di bawah kurva precision–recall satu kelas.
- **mAP**: rata-rata AP seluruh kelas.
- **mAP@0,5** (PASCAL VOC) vs **mAP@[0,5:0,95]** (COCO: rata-rata 10 ambang IoU, lebih ketat).

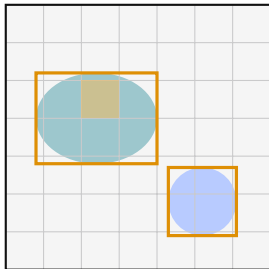
Detektor Dua Tahap: Keluarga R-CNN

Usulkan Region, Lalu Klasifikasikan



Model	Sumber proposal	Ide kunci	Kecepatan
R-CNN (2014)	Selective search	CNN dijalankan per region ($\sim 2000 \times$)	~ 47 s/citra
Fast R-CNN (2015)	Selective search	CNN sekali per citra + RoI pooling	~ 2 s/citra
Faster R-CNN (2015)	Region Proposal Network	Proposal dipelajari dari <i>anchor boxes</i>	$\sim 0,2$ s/citra

- **Anchor**: box acuan berbagai skala & rasio di tiap lokasi feature map; RPN memprediksi *objectness* + pergeseran box.
- Akurat, terutama untuk objek kecil, namun lebih lambat karena dua tahap.



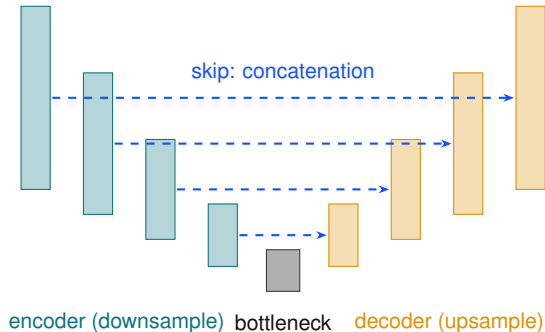
Sel oranye memuat pusat objek $\Rightarrow$ sel itu yang bertanggung jawab memprediksinya.

Deteksi sebagai satu masalah regresi

Citra dibagi grid $S \times S$. Tiap sel memprediksi B box (x, y, w, h, conf) dan C probabilitas kelas:

$$\text{output} = S \times S \times (5B + C) = 7 \times 7 \times 30 \quad (\text{VOC: } B=2, C=20)$$

- **Satu forward pass**, tanpa tahap proposal $\Rightarrow \sim 45$ FPS (real-time) pada YOLOv1.
- Versi awal lemah pada objek kecil & berdempetan; versi modern (YOLOv5–v11) memakai anchor/anchor-free, FPN, dan backbone CNN efisien.
- Trade-off umum: **satu tahap = cepat, dua tahap = akurat.**



FCN (Long et al., 2015)

- Ganti FC dengan conv $1 \times 1 \Rightarrow$ bisa menerima ukuran input apa pun.
- *Upsampling* (transposed conv) memulihkan resolusi: output $H \times W \times C$.

U-Net (Ronneberger et al., 2015)

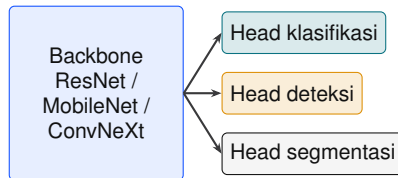
- Encoder mengompres konteks, decoder memulihkan detail.
- **Skip concatenation** mengirim fitur resolusi tinggi dari encoder ke decoder $\Rightarrow$ batas tajam.
- Standar de facto citra medis & model difusi.

Mask R-CNN (He et al., 2017)

Faster R-CNN + **cabang mask**: untuk setiap RoI, prediksi mask biner 28×28 per kelas, paralel dengan kelas & box.

- **RoIAlign**: interpolasi bilinear tanpa pembulatan koordinat (RoI pooling membulatkan $\Rightarrow$ mask bergeser).
- Loss total: $L = L_{cls} + L_{box} + L_{mask}$.

Benang merah hari ini:



Arsitektur yang dipelajari di Bagian 1–4 adalah “mesin fitur” untuk seluruh tugas computer vision; torchvision menyediakan `fasterrcnn_resnet50_fpn`, `maskrcnn_resnet50_fpn`, dan `deeplabv3_resnet50`.

- 1 **AlexNet** → **VGG**: ReLU, dropout, augmentasi, GPU; lalu tumpukan 3×3 seragam (RF sama, bobot $18C^2$ vs $25C^2$).
- 2 **Efisiensi awal**: konvolusi 1×1 dan GAP (GoogLeNet) membuang FC raksasa yang memakan $\sim 90\%$ parameter VGG.
- 3 **ResNet**: $H(x) = F(x) + x$ mengatasi *degradation problem*; suku $+1$ menjaga aliran gradien; bottleneck $1 \times 1 - 3 \times 3 - 1 \times 1$.
- 4 **MobileNet**: depthwise + pointwise memangkas biaya menjadi $\frac{1}{N} + \frac{1}{D_K^2}$ ($\sim 9\times$ lebih hemat); EfficientNet menskalakan d, w, r bersamaan.
- 5 **ConvNeXt**: resep training modern, patchify stem, depthwise 7×7 , inverted bottleneck, GELU, LayerNorm $\Rightarrow$ CNN murni menyaingi Swin.
- 6 **Deteksi & segmentasi**: format box (VOC/COCO/YOLO), IoU, NMS, mAP; dua tahap (Faster R-CNN) vs satu tahap (YOLO); encoder-decoder (FCN, U-Net); Mask R-CNN.

Pertemuan 6 — Studio Proyek 2: Fine-tuning CNN Pretrained. Bawa laptop, siapkan akun Kaggle/Colab dengan GPU; kelompok 2 orang.

- 1 Blok 3 VGG-16:** input $56 \times 56 \times 128$, tiga conv 3×3 256 filter ($P = 1$) lalu max-pool 2×2 . Hitung ukuran output dan total parameter (termasuk bias).
- 2 ResNet pada 512 kanal:** bandingkan jumlah bobot basic block ($2 \times$ conv 3×3 , 512 kanal) dengan bottleneck (1×1 128, 3×3 128, 1×1 512). Berapa kali lipat penghematannya?
- 3 Depthwise separable 5×5 :** dengan $M = N = 256$, turunkan rasio biaya terhadap konvolusi standar. Mengapa penghematannya lebih besar dibanding kernel 3×3 ?
- 4 IoU:** ground truth (10, 20, 50, 80) dan prediksi (30, 40, 70, 90) dalam format (x_1, y_1, x_2, y_2) . Hitung IoU; apakah TP pada ambang 0,5? Tulis juga prediksi dalam format COCO dan YOLO (citra 100×100).
- 5 Analisis tabel perbandingan:** pilih dua backbone untuk aplikasi deteksi penyakit daun di ponsel petani, dan justifikasi dengan angka params, GFLOPs, dan top-1.

Kumpulkan jawaban tulisan tangan (foto/PDF) melalui LMS sebelum Pertemuan 6.

-  A. Krizhevsky, I. Sutskever, G. Hinton. ImageNet Classification with Deep Convolutional Neural Networks. *NeurIPS*, 2012.
-  K. Simonyan, A. Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. *ICLR*, 2015.
-  C. Szegedy et al. Going Deeper with Convolutions. *CVPR*, 2015.
-  S. Ioffe, C. Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *ICML*, 2015.
-  K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016.
-  G. Huang, Z. Liu, L. van der Maaten, K. Weinberger. Densely Connected Convolutional Networks. *CVPR*, 2017.
-  A. Howard et al. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv:1704.04861*, 2017.
-  M. Sandler et al. MobileNetV2: Inverted Residuals and Linear Bottlenecks. *CVPR*, 2018.
-  M. Tan, Q. Le. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML*, 2019.

-  Z. Liu et al. A ConvNet for the 2020s. *CVPR*, 2022.
-  R. Girshick et al. Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation. *CVPR*, 2014.
-  R. Girshick. Fast R-CNN. *ICCV*, 2015.
-  S. Ren, K. He, R. Girshick, J. Sun. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *NeurIPS*, 2015.
-  J. Redmon et al. You Only Look Once: Unified, Real-Time Object Detection. *CVPR*, 2016.
-  J. Long, E. Shelhamer, T. Darrell. Fully Convolutional Networks for Semantic Segmentation. *CVPR*, 2015.
-  O. Ronneberger, P. Fischer, T. Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation. *MICCAI*, 2015.
-  K. He, G. Gkioxari, P. Dollár, R. Girshick. Mask R-CNN. *ICCV*, 2017.

-  M. Everingham, L. Van Gool, C. Williams, J. Winn, A. Zisserman. The PASCAL Visual Object Classes (VOC) Challenge. *IJCV*, 2010.
-  T.-Y. Lin et al. Microsoft COCO: Common Objects in Context. *ECCV*, 2014.
-  PyTorch Team. torchvision.models — Models and pre-trained weights.
<https://pytorch.org/vision/stable/models.html>
-  PyTorch Team. torchvision.ops.box_convert.
https://pytorch.org/vision/stable/generated/torchvision.ops.box_convert.html
-  Ultralytics. YOLO Dataset Format (Object Detection).
<https://docs.ultralytics.com/datasets/detect/>

IF25-40401 · DEEP LEARNING

Terima Kasih

Pertemuan Berikutnya: Studio Proyek 2 — Fine-tuning CNN Pretrained pada ImageNet

Tim Dosen Deep Learning · Institut Teknologi Sumatera (ITERA)